VESPER · Guion de presentación

Fernando · Enzo · Marcelo
Desarrollo Fullstack II · Segunda entrega
Duración: 15 minutos · 5 minutos por integrante

El tiempo de cada bloque incluye la exposición y la demostración. Las indicaciones en cursiva sirven como apoyo y no se leen en voz alta.

1. Fernando — Introducción, React y tienda pública

Tiempo: 0:00–5:00

> *Deja abierta la página principal.*

Buenos días. Somos Fernando, Enzo y Marcelo, y presentamos la segunda entrega de Vesper, nuestra tienda de ropa inspirada en cultura pop.

En la primera entrega teníamos páginas construidas con HTML, CSS y JavaScript. En esta etapa migramos la aplicación a React, conservando la identidad visual y los productos, e incorporamos nuevas vistas, persistencia y pruebas.

Organizamos el proyecto en componentes reutilizables, páginas, un contexto compartido y un servicio que administra los datos. Esta separación permite trabajar cada responsabilidad de forma independiente.

> *Mostrar el catálogo y abrir un producto.*

Por ejemplo, cada producto utiliza el componente `ProductCard`. Este recibe mediante props los datos del producto y una función para agregarlo al carrito.

Así podemos utilizar la misma tarjeta en el catálogo, los resultados de búsqueda y las ofertas. Cuando cambian los datos recibidos, React actualiza lo que se muestra.

> *Buscar “Twin Peaks”, cambiar una categoría y entrar a Ofertas.*

La búsqueda y el filtro de categoría se pueden combinar. Estos criterios quedan representados en la dirección de navegación, lo que permite conservarlos al compartir o volver a una vista.

También implementamos renderizado condicional: mostramos un mensaje cuando no hay coincidencias, una etiqueta cuando existe descuento y deshabilitamos la compra cuando un producto está agotado.

> *Mostrar brevemente `src/components/ProductCard.jsx`.*

Aquí se pueden identificar las props del componente y el evento del botón. El componente muestra la información y comunica la acción; las reglas del carrito se resuelven en el servicio de datos.

> *Activar la vista móvil del navegador.*

Para el diseño responsivo utilizamos Bootstrap, complementado con nuestros estilos. En escritorio se muestran varias tarjetas por fila; en pantallas pequeñas se reorganizan y la navegación se convierte en un menú desplegable.

Además del catálogo incorporamos categorías, ofertas, detalle de producto, nosotros, blog y contacto. La tienda se puede explorar sin iniciar sesión.

Cierre de Fernando:

Ahora Enzo mostrará cómo conectamos estas vistas con los datos y cómo funciona el proceso de compra y administración.

2. Enzo — Persistencia, compra y administración

Tiempo: 5:00–10:00

> *Deja preparado un carrito con un producto.*

La aplicación utiliza una base de datos simulada en JavaScript, según el alcance de esta entrega. Los productos, usuarios, categorías, pedidos y carrito se guardan en Local Storage.

La sesión se mantiene en Session Storage, por lo que corresponde a la pestaña del navegador. Estos datos son locales y no se sincronizan entre equipos.

Las operaciones están centralizadas en `src/services/store.js`. Ahí implementamos las validaciones y las funciones para crear, consultar, actualizar y eliminar registros.

> *Cambiar la cantidad de un producto en el carrito.*

El carrito calcula los subtotales y valida las cantidades. No permite superar el stock disponible ni el límite de veinte unidades por producto.

Después de una operación, el servicio guarda los cambios y notifica a React. El contexto compartido permite que distintas vistas, como el carrito y su contador, reflejen el mismo estado.

> *Entrar al checkout y completar datos ficticios.*

En el checkout solicitamos los datos de contacto y el método de entrega. El usuario puede elegir despacho o retiro, y el resumen actualiza el costo correspondiente.

Cuando una persona inicia sesión y tiene su dirección guardada en el perfil, esos datos se completan automáticamente.

> *Seleccionar pago rechazado y confirmar.*

El pago es una simulación y no realiza cobros. Incluimos una opción de rechazo para demostrar el manejo de errores.

En este caso no se genera ningún pedido, no se descuenta stock y los productos permanecen en el carrito.

> *Reintentar con pago aprobado.*

Cuando la simulación se aprueba, se crea el pedido, se descuenta el stock y se vacía el carrito.

El comprobante conserva el nombre, precio y cantidad de los productos comprados. De esa manera, una modificación posterior del catálogo no cambia los datos históricos del pedido.

> *Ingresar como administrador y mostrar Productos, Categorías y Usuarios.*

El panel administrativo permite gestionar productos, categorías y usuarios. También contiene pedidos, comprobantes y reportes.

Podemos modificar precios, existencias y descuentos. Existen validaciones para evitar datos incorrectos, correos duplicados y acciones como eliminar nuestra propia cuenta activa.

> *Mostrar Reportes.*

Los reportes resumen las ventas simuladas y permiten identificar productos con stock crítico, definido como cinco unidades o menos.

El control por roles sirve para esta demostración frontend. En una aplicación real, la autenticación y autorización también deberían validarse en un servidor.

Cierre de Enzo:

Marcelo explicará cómo verificamos estos comportamientos y qué resultados obtuvimos con las pruebas.

3. Marcelo — Pruebas, resultados y cierre

Tiempo: 10:00–15:00

> *Deja abierto el editor y una terminal en el proyecto.*

Para verificar el funcionamiento configuramos Jasmine y Karma. Jasmine permite definir los casos y sus expectativas; Karma ejecuta esas pruebas en un navegador real.

La suite final contiene cincuenta y seis pruebas aprobadas. Incluye componentes React, formularios, navegación, administración y reglas del servicio de datos.

> *Abrir `tests/components.spec.jsx` y mostrar un caso sencillo.*

En los componentes comprobamos cuatro aspectos principales: renderizado, propiedades, estado y eventos.

Por ejemplo, verificamos que una tarjeta muestre los datos recibidos, que un clic invoque la función correspondiente y que un producto sin stock tenga su botón deshabilitado.

También probamos mensajes condicionales, filtros y formularios con entradas válidas e inválidas.

> *Mostrar un espía de Jasmine y un caso de `tests/store.spec.js`.*

Utilizamos espías y respuestas simuladas para aislar comportamientos. Así podemos comprobar cómo responde una interfaz cuando el servicio devuelve un error.

Por separado probamos el servicio real con almacenamiento en memoria. Un caso importante verifica que un pago rechazado conserve el carrito, el stock y los pedidos sin modificaciones.

> *Ejecutar `npm test` y mostrar el resultado.*

Nuestro proceso consiste en definir el comportamiento esperado, preparar los datos, ejecutar una acción y comprobar el resultado. Si aparece un defecto, lo corregimos y repetimos las pruebas.

La ejecución final aprobó los cincuenta y seis casos. La cobertura medida fue de un setenta coma seis por ciento de líneas, considerando todos los módulos JavaScript y JSX de la aplicación, salvo el archivo que monta la raíz de React.

Esto no significa que la aplicación esté libre de errores. El informe identifica las áreas que tienen menor cobertura y las limitaciones de las pruebas.

> *Mostrar el informe de testing y una captura móvil.*

Complementamos las pruebas con cuarenta y ocho comprobaciones en navegador sobre los recorridos de compra, perfiles, administración y diseño responsivo.

Revisamos tamaños de móvil, tableta y escritorio. Estas comprobaciones son independientes y no aumentan el porcentaje de cobertura unitaria.

> *Mostrar el ERS.*

Finalmente, actualizamos el ERS con los requisitos y criterios de aceptación, y documentamos el alcance del testing.

Utilizamos inteligencia artificial como apoyo, conforme a la autorización de la actividad. Revisamos el código y comprobamos sus resultados; por eso podemos explicar tanto su funcionamiento como sus límites.

Con esta entrega, Vesper incorpora una estructura en React, navegación, persistencia local, compra simulada, administración y pruebas automatizadas. Muchas gracias.

Preparación antes de presentar

Ensayen las transiciones y dejen preparadas las pestañas, las cuentas de demostración y los archivos. Los tres deben poder explicar por qué usan props, cómo se actualiza el estado y qué ocurre cuando falla un pago.

Integrante	Preparar antes de su bloque
Fernando	Inicio, catálogo, detalle de producto, `ProductCard.jsx` y vista móvil.
Enzo	Carrito con un producto, datos ficticios para checkout y cuenta de administrador.
Marcelo	Terminal en el proyecto, pruebas de componentes y servicio, informe de testing y ERS.

Cuenta de administrador: `admin@vesper.cl` / `admin123`
Cuenta de cliente: `usuario@vesper.cl` / `user123`

Los resultados citados corresponden a la versión verificada de esta entrega. Si modifican el código, vuelvan a ejecutar las pruebas antes de presentar.